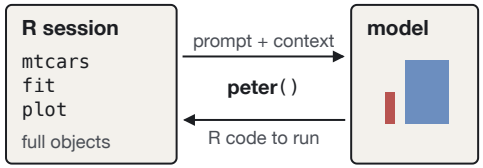
# Agents in your R session with gptr :: CHEATSHEET

`peter()` runs language model agents in your current R session, on objects already in memory. The name honors Peter Naur's work on readable language definitions and Peter Cathcart Wason's research with Jonathan Evans on reasoning. In Naur's spirit, gptr records sessions as runnable documents.

## How it works




R computes on the full objects, while the model receives size-limited descriptions, previews and tool output that may include data values (`?gptr_egress`).

**gptr_describe**(`mtcars`)
What a model is shown of an object.

**peter**(`q, .opts = list(context = "none")`)
Leave out the workspace listing, your recent changes and code, and object descriptions.

## Set up

**pak::pak**(`"Broccolito/gptr@main"`)
**library**(`gptr`)
Install gptr 1.0 from GitHub (R 4.2 or later).

**gptr_env**(`"~/keys/.env"`)
Load keys from a .env file; gptr redacts them.

**gptr_login**(`"openrouter"`)
Sign in with OAuth or a masked key.

**gptr_providers**(`);` **gptr_models**(`"sonnet"`)
List the routes; search the offline model catalog.

**gptr_config**(`model = opus, .scope = "user"`)
Save a default model (session, project or user).

**gptr_init**(`"."`)
Create `.gptr/`, so sessions and recorded answers outlive R.

**gptr_trust**(`".", trust = TRUE`)
Trust the project's settings, skills and plugins.

**gptr_permissions**(`allow = "r(fn:saveRDS)"`)
Add a rule for this R session, or list the rules.

`e =` **gptr_config**`()$egress; e$openai = "ack"`
**gptr_config**(`egress = e, .scope = "user"`)
Let unattended runs send to openai.

No model set? At the console `peter()` offers the Claude Code or Codex CLI; `peter("...")` uses an Anthropic, OpenAI or Google key, else a CLI.

## Ask Peter from R

`peter("...")` returns a session, in scripts, loops and functions too. Where nobody can approve (Rscript, knitr), use `mode = auto` or an allow rule, or the first change stops the run.

`s =` **peter**(`"Fit mpg on wt; save as fit.", mtcars`)
The prompt first, then the objects to work on.

**s$text; s$value; s$turns**
The last answer, the newest `gptr_return()` value, the prompt count.

**s$usage; s$cost; s$envir**
Tokens, cost (NA without prices), the workspace.

`col = "mpg";` **peter**(`"Plot {col} by cyl."`)
A literal prompt interpolates `{name}` from R.

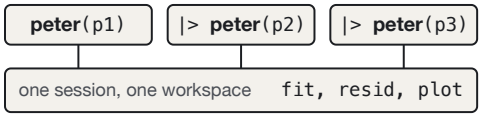
**peter**(`"Clean it.", raw, mode = auto, model = "openai/gpt-6-sol"`)
Name other arguments exactly; bare names work.

**peter**(`"Summarize.", sales, budget = list(tokens = 5e4, turns = 5)`)
Limits for one call; `turns` counts model requests.

**gptr_return**(`coef(fit))  # in agent R code`
Mark the result, `s$value` until a later one.

**tryCatch**(**peter**(`"Fit.", d), gptr_error = function(e) e$session`)
A stopped run carries its session; after Esc or Ctrl-C, `gptr_last()`.

## Pipe: one session, many turns




`s |>` **peter**(`"Now plot the residuals."`)
One more turn, same session and workspace.

`s |>` **peter**(`"Add a caveat.", model = opus`)
Switch the model; the history is kept.

`b =` **gptr_fork**`(s)`
`b |>` **peter**(`"Try a log model instead."`)
Branch off; only in-place edits, such as `:=`, reach the original.

`s;` **summary**`(s);` **gptr_last**`()`
Print it, its history, the most recent session.

## The console

**session** = **peter**`()`
`session |>` **peter**`()`
Open the console; `/exit` returns the session.

### AT THE peter> PROMPT

| | |
|---|---|
| `Fit mpg on wt.` | A prompt for the session |
| `!head(d)` | Run R and show the model |
| `!!x = 2` | Run R; send no code or output |
| ```` ```r … ``` ```` | Fenced R, run like `!code` |
| `"""` … `"""` | A multi-line prompt |
| `line \` | Continue on the next line |
| `@notes.md` | Attach a file's first 40 lines |
| `@fit` | Attach an object |
| `{n}` | Insert an R value |

### SLASH COMMANDS

| | |
|---|---|
| `/help [cmd]` | Commands and syntax |
| `/model /mode` | Show or switch either |
| `/status` | Session, model, mode |
| `/cost /env` | Tokens and cost; objects |
| `/tools` | Tools and helpers |
| `/compact` | Compact the history |
| `/clear` | New chat, same objects |
| `/resume /fork` | Resume or fork a session |
| `/doc [path]` | The history document |
| `/skills /mcp` | List skills, MCP servers |
| `/<name>` | A `.gptr/prompts/` template |
| `/permissions` | Show or change rules |
| `/undo /redo` | Undo or redo a turn |
| `/retry` | Resend the last prompt |
| `/rewind [k]` | Keep turns 1 to k |
| `/exit` | Leave; return the session |

**gptr_doc**(`"log.R"`)
Record the console turns and `!code` as a runnable `.R` transcript.

### WHILE AN ANSWER RUNS

In a terminal (not Windows) type, then Enter steers and Tab follows up. Ctrl-C (RStudio: Esc) pauses: **[s]**teer, **[f]**ollow-up, **[c]**ontinue, **[a]**bort, **[b]**ackground. Approvals: **[y]**es, **[a]**lways (session), **[p]** always (project), **[n]**o (`n <text>` says what to do instead).

## System 1 and System 2

**System 1** typed decision → `TRUE "lung" 1.88` vector + probabilities

**System 2** agent with tools → `gptr_session` $text, $value, objects

The model decides what `peter()` returns. A decision model (Jev, Clef) is sent each input and answers once per input; a conversational model works in turns.

### SYSTEM 1: TYPED DECISIONS

`rct =` **peter**`("Is this an RCT?", abstracts, model = jev)`
Logical: TRUE when P(yes) ≥ `threshold` (0.5).

**gptr_prob**(`rct`)
P(yes) for each element.

**peter**(`"Which tissue?", cells, model = jev, choices = c("liver", "lung", "other")`)
One label per input.

**peter**(`"How severe?", events, model = jev, levels = c("low", "mid", "high")`)
A score: the expected level from 0, such as 1.88.

**peter**(`q, x, model = jev, min_confidence = 0.8`)
Below it (|2p−1| for yes or no) an answer is NA, or what `uncertain` says.

`abstracts[rct];` **sum**(`rct`)
Plain vectors for subsetting, `sum()`, `if` and `for`.

`s |>` **peter**(`"Did it succeed?", model = jev`)
Judge a session's last answer; adds no turn.

### SYSTEM 2: AGENTS

**peter**(`"Plan it.", d, model = opus, .opts = list(thinking = "high")`)
More reasoning, `off` to `max`, or a suffix such as `opus:high`.

## Try it offline

```
step = list(tool = "r", input = list(
  code = "fit = lm(mpg ~ wt, mtcars)"))
fake = gptr_fake_provider(
  list(step, "Done."))
s = peter("Fit mpg on wt.", mtcars,
  model = fake, mode = auto); coef(fit)
```

The first reply runs R; the second is `s$text`.

## Models and providers

```
"anthropic/claude-sonnet-5-5:high"
```
provider · model id · thinking

Aliases work too: `sonnet`, `opus`, `haiku`, `gpt`, `flash`, `jev`, `claude_code`, `codex`.

| | |
|---|---|
| `anthropic/…` | `ANTHROPIC_API_KEY` |
| `openai/…` | `OPENAI_API_KEY` |
| `google/…` | `GEMINI_API_KEY` |
| `openrouter/…` | `OPENROUTER_API_KEY` |
| `azure/…` | `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT` |
| `bedrock/…` | `AWS_BEARER_TOKEN_BEDROCK`, `AWS_REGION` |
| `ollama/…` | Local, no key; also lmstudio, vllm |
| `claude-cli` | `claude auth login` |
| `codex` | `codex login` |
| `jev` | System 1: `TYPESAFE_API_KEY` |
| `ollama/clef` | System 1: local, Ollama 0.35.1+ |

**gptr_models**(provider = "ollama", refresh = TRUE)
List the models Ollama has installed.

m = **list**(id = "my-id", tool_call = TRUE)
**gptr_config**(providers = list(bedrock = list(models = list(m))))
Declare a model for `model = "bedrock/my-id"`; each call replaces `providers`.

**gptr_register**(**gptr_provider**("corp", "openai-completions", base_url = u, auth = "CORP_KEY", models = list(m)))
Add any OpenAI-compatible endpoint; registering the name again replaces it.

## Permissions and trust

| | |
|---|---|
| `plan` | Read only; next call gets the plan |
| `manual` | Default: reads run, changes ask |
| `edits` | Also file edits in the project |
| `auto` | Asks only for critical actions and secret reads |
| `yolo` | Nearly all; critical actions still ask |

**gptr_permissions**(deny = "r(sh:rm *)")
Rules: `deny` beats `ask` beats `allow`.

**gptr_rewind**(s)
Undo the last turn: objects, files, chat.

`auto` and `yolo` refuse a `system()` command the code computes (use `peter$sh()`). The gate is advisory, not a sandbox: `?gptr_security`.

## Record, replay and export

Bind a document; gptr writes eligible top-level calls and their executed R code into it, with model and execution metadata.

```
peter("Fit mpg on wt.")
# >>> gptr:46d369 model=…
fit = lm(mpg ~ wt, mtcars)
coef(fit)[["wt"]]
#> [1] -5.344472
## Decision: linear fit
# <<< gptr:46d369
```
prompt · header · code · output · note · end

**gptr_doc**("analysis.R")
Record into this `.R`, `.Rmd`, `.qmd` or `.ipynb`; to knit or render, bind in the setup chunk.

**gptr_source**("analysis.R", replay = "replay")
Rerun with no model call; `"live"` asks again.

`GPTR_REPLAY=replay Rscript analysis.R`
Prove from the shell that a script needs no model.

**gptr_blocks**("analysis.R")
Block status: fresh, stale, user-edited or undone.

**gptr_doc**("report.Rmd")
**rmarkdown::render**("report.Rmd")
Render as usual; blocks become `gptr-<id>` chunks.

**gptr_doc**("analysis.ipynb", sync = TRUE)
Write the blocks queued while it was open.

**gptr_doc**(FALSE)
Stop recording.

Replay runs the recorded code and needs its inputs and packages; a new R session gets `s$text` back only from `.gptr/`. A top-level `gptr_coach()` call replays too.

### EXPORT A RECORDED .R

Export needs fresh blocks and literal arguments (write `mode = "auto"`), with data objects only as names after the prompt.

**gptr_export**("analysis.R", "plain.R")
Plain R: recorded code runs, `peter()` calls become comments.

**gptr_export**("analysis.R", "peter.R", format = "peter")
`peter()` calls run; recorded code is commented.

## Learn more

Guides at broccolito.github.io/gptr/articles, from Getting started to Harnesses and coaching, and the function reference.

## Tools, files and artifacts

The model calls `read`, `r`, `edit`, `write` and `ask`; in `r` it uses `peter$` helpers, which you can call too.

**peter**("Review R/.", tools = extended)
Presets: `minimal`, `standard`, `readonly` (`r` can still write files), `extended` (adds grep, find, ls).

**peter**(q, tools = c("-write", "+grep"))
Remove or add single tools.

**peter$read**("R/fit.R", limit = 40)
**peter$grep**("lm\\(", path = "R")
**peter$find**("*.R"); **peter$ls**("R")
Read, search, find and list files.

**peter$edit**("R/fit.R", list(list(oldText = "n=10", newText = "n=20")))
**peter$write**("notes.txt", "done\n")
Edit text in place; write files atomically.

**peter$sh**(c("git", "status"))
**peter$bg**(c("Rscript", "long.R"))
Run a program, or one in the background.

**peter$sql**("SELECT cyl, AVG(mpg) FROM mtcars GROUP BY cyl", mtcars)
SQL on data frames; `peter$py()` runs Python.

**peter**("Make a Shiny app of it.", survey)
An artifact: `app.R`, launched at the console.

**gptr_artifacts**()
**gptr_artifacts**("survey-view", open = TRUE)
List apps; open, stop or relaunch a version.

## Skills, plugins and MCP

**gptr_skills**()
Skills of the project, user, plugins and packages.

**peter**("Speed up f.", f, skills = "high-performance-r")
Preload a skill; write yours as `<name>/SKILL.md` in `.gptr/skills/`.

**peter**("Report units.", plugins = "./lab")
Enable a plugin; `gptr_plugins()` lists them.

**gptr_mcp_add**("docs", url = "http://127.0.0.1:8000/mcp")
Add an MCP server, or `command =` for stdio.

**peter$mcp$<server>$<tool>**(...)
MCP tools are R functions the agent calls.

h = **gptr_mcp_serve**(); **h$config**
**h$stop**()
Serve this R session to Claude Code or Codex.

up = **gptr_tool**("shout", "Upper-case.", fun = function(x) toupper(x))
off = **gptr_register**(up)
Add a tool; `off()` removes it.

## Teams, fan-outs, background

Members inherit the mode. A team fails only when no member finishes.

s = **peter**("Review fit.", agents = list(stats = agent(system = "Check stats."), code = agent(system = "Check code.")))
A team of named roles; `s$stats$text` reads one.

f = **peter**("Summarize; gptr_return() it.", cohorts, parallel = 2)
One child per element; values in `f$value`.

**gptr_parallel**(a = **peter**("Plan.", envir = new.env()), b = **peter**("Test.", envir = new.env()))
Run calls at once, unrecorded; `envir` keeps their objects apart.

b = **peter**("Check.", background = TRUE)
**gptr_wait**(b)
Experimental: return at once; also `gptr_steer()`, `gptr_cancel()`, `gptr_jobs()`.

**gptr_usage**(s, by = "agent")
Tokens and cost by session, agent, model or route.

## Tailor and coach a harness

A harness is `peter()` with your arguments, rules and coaching (a note, skills, a review) built in, never loosening permission checks. `gptr_coach()` runs unattended R processes: most tasks need `mode = auto`.

h = **gptr_tailor**(peter, mode = auto, coaching = list(note = "Give units."))
A call's own arguments win; its tools add.

h = **gptr_tailor**(h, rules = list(never = list(list(rule = "r(fn:stepAIC)", why = "No stepwise selection."))))
Refused in every mode; `require` asks for a step.

s = **h**("Fit mpg on wt.", mtcars); **s$text**
Call it exactly like `peter()`, pipes included.

q = "Average x; gptr_return() it."
h = **gptr_coach**(q, x, harness = h, check = function(r) isTRUE(r$value == 2))
New coaching is kept only if it passes `check`.

h = **gptr_coach**(q, x, harness = h, coach = claude_code)
Coach on a Claude Code plan (or `codex`).

**saveRDS**(h, "h.rds")
h = **gptr_tailor**(readRDS("h.rds"))
Share it; loading checks it and shows any code.

**gptr_tailor**(h, dir = "my-harness")
Write it as Markdown; gptr never reads it back.